PLAGIARISM SCAN REPORT

Date	December 16, 2025
Exclude URL:	NO

Unique Content	**100**	Word Count	916
Plagiarized Content	**0**	Records Found	0

CONTENT CHECKED FOR PLAGIARISM:

Data Structures And Analysis

Final Project Report

Project Name: Cybershooter

Date: 16/12/2025

Version: Final Report

Report By: Samra Mehmood

Course: BEE 61 C

Submitted to: Lt Col Muhammad Imran Javed

Revisions

Version	Primary Author(s)	Abstract	Date Completed
Final	Samra Mehmoood Abdullah Khan	CyberShooter is a 2D arcade-style shooting game developed using Python and the Pygame library, designed to demonstrate the practical application of Data Structures and Algorithms (DSA) in real-time software. The project integrates core DSA concepts like Stack, Queue and Doubly Linked List directly into gameplay mechanics such as bullet handling, enemy management, background rendering and visual effects. The final system provides smooth gameplay, an interactive user interface, sound integration and efficient memory management, fulfilling all academic and functional objectives of the course.	16/12/2025

Contents

1. Introduction

1.1 Purpose

The purpose of this project is to design and implement a functional game that applies fundamental data structures in a real-world scenario. CyberShooter demonstrates how abstract DSA concepts can be used beyond theory, particularly in game development and event-driven systems.

1.2 Scope

CyberShooter is a single player, 2D shooting game where the player controls a futuristic jet to destroy incoming

enemies while avoiding collisions. The project scope includes player movement, shooting mechanics, enemy spawning, collision detection, scoring, power-ups, animations, sound effects and background effects.

1.3 Objectives

• Apply Stack, Queue, and Doubly Linked List in a practical system

• Develop a complete playable game using Python

• Demonstrate clean, modular and maintainable code

• Ensure smooth gameplay and responsive controls

2. System Overview

CyberShooter runs as a standalone desktop application. The system is event-driven and frame-based, running at 60 FPS. Game entities such as bullets, enemies, explosions, stars and visual effects are managed using custom DSA implementations rather than Python built-in lists.

3. Data Structures Used and Their Application

3.1 Doubly Linked List (DLL)

A custom Doubly Linked List is implemented as the base data structure. It supports efficient insertion and removal of nodes from both ends and from the middle of the list.

Used For:

• Background stars

• Bullet trail effects

• Static visual sparks

• Internal implementation of Stack and Queue

Justification: DLL allows efficient deletion of dynamic objects during gameplay without excessive shifting operations, which is essential for performance.

3.2 Stack (LIFO)

The Stack is implemented using the Doubly Linked List.

Used For:

• Bullet management

Why Stack?

• Bullets are fired and removed frequently

• Latest bullets are processed first

• Supports arbitrary removal during collisions

This reflects the Last-In-First-Out behaviour naturally.

3.3 Queue (FIFO)

The Queue is also implemented using the Doubly Linked List.

Used For:

• Enemy management

• Power-ups

• Explosions

• Planets in the background

Why Queue?

• Objects are processed in the order they appear

• Oldest enemies and effects are removed first

• Matches real-time event sequencing

4. Function Requirements

• The system shall allow the player to move left and right using keyboard input

• The system shall allow the player to shoot bullets

• The system shall spawn enemies dynamically

• The system shall detect collisions between bullets and enemies

• The system shall manage player lives and score

• The system shall provide power-ups (extra life, double fire)

• The system shall display a game over screen and allow restart

5. Non- Functional Requirements

5.1 Performance

• Runs at 60 FPS on a standard PC

• Real-time collision detection with minimal delay

5.2 Usability

• Simple keyboard controls

• Clear HUD and visual indicators

5.3 Reliability

• Stable execution without crashes

• Safe handling of missing assets

5.4 Portability

• Runs on any OS supporting Python and Pygame

6. Game Design and Flow

6.1 Game Flow

1. Intro Screen

2. Player starts the game

3. Enemies spawn continuously

4. Player shoots and avoids collisions

5. Score increases with enemy destruction

6. Game ends when lives reach zero

7. Restart or exit option

6.2 Controls

• Left Arrow: Move left

• Right Arrow: Move right

• Space: Shoot

• P: Pause

• ESC: Exit

7. User Interface

• Neon-themed HUD

• Score, high score, lives and power-up timer display

• Animated background with stars and planets

• Visual effects such as glow, explosions and trails

8. Challenges Faced

• Integrating custom data structures with real-time rendering

• Efficient removal of objects during collisions

• Managing performance with multiple dynamic elements

• Debugging graphical and collision-related issues

9. Results and Achievements

• Successfully implemented Stack, Queue and DLL

• Fully playable game with advanced UI

• Smooth animations and effects

• Clean and modular code structure

10. Future Enhancements

• Multiplayer mode

• Advanced enemy AI

• Additional power-ups

• Enhanced sound design

• Level-based progression

11. Conclusion

The CyberShooter project successfully demonstrates how Data Structures and Algorithms can be applied in a real-world software system. By integrating Stack, Queue and Doubly Linked List into gameplay mechanics, the project bridges the gap between theoretical knowledge and practical implementation.

The final system meets all academic requirements and serves as a strong foundation for future enhancements.

12. References

• Python Official Documentation - 3.14.2 Documentation

• Pygame Documentation - Pygame Front Page — pygame v2.6.0 documentation

• IEEE Software Requirements Specification (SRS) Standards

MATCHED SOURCES: